SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1401 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom-up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

Duration :60 Mins
On Class
QUESTIONS: 25 **Total Marks:50**

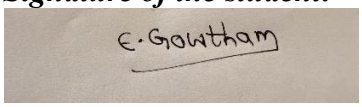
Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Enamala Gowtham/192311190 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the token stream:
id + id * id
Explain how the parser validates the syntactic structure of the expression.

Ans) Lexical analyzer produces id + id * id
The parser checks whether the token sequence follows the grammar of expressions.
For example:
id + id * id
The parser recognizes:

- id as an operand.
- + and * as operators.
- * has higher precedence than +.

So, the structure is:
id + (id * id)
Thus, the expression is **syntactically valid**.

Q2. Given the grammar:
S → aSb | ε
Determine whether the string aaabbb belongs to the language using derivation.

Ans) Grammar S → aSb | ε, string aaabbb
Given:
S → aSb | ε
Derivation:
S
⇒ aSb
⇒ aaSbb
⇒ aaaSbbb
⇒ aaaεbbb
⇒ aaabbb
Therefore, **aaabbb belongs to the language.**

Q3. Remove left recursion from the grammar:
E → E + T | T

Ans) Remove left recursion
Given:
E → E + T | T
The left-recursion-free grammar is:
E → T E'
E' → + T E' | ε

Q4. Find FIRST(A) for the grammar:
A → aB | ε
B → b

Ans)Find FIRST(A)

Given:
A → aB | ε
B → b
Therefore:
FIRST(A) = { a, ε }

Q5. Determine whether the grammar is suitable for top-down parsing:
A → Aa | b

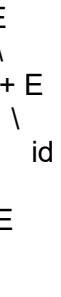
Ans) Is A → Aa | b suitable for top-down parsing?
No, the grammar contains **immediate left recursion**:
A → Aa
Top-down parsers such as recursive descent parsers cannot directly handle left recursion because it may cause **infinite recursion**.
Therefore, the grammar is **not suitable for top-down parsing**.

Q6. Apply left factoring to the grammar:
S → while E do S | while E do S else S

Ans) Apply left factoring
Given:
S → while E do S | while E do S else S
Common prefix:
while E do S
After left factoring:
S → while E do S S'
S' → else S | ε

Q7. Construct a parse tree for the expression:
id * id + id
using the grammar:
E → E + E | E * E | id

Ans) Parse tree for id * id + id
Grammar:
E → E + E | E * E | id
Taking * with higher precedence than +:



Expression represented:
(id * id) + id
The given grammar is ambiguous, but this tree represents the usual precedence interpretation.

Q8. Write the recursive descent procedures for the grammar:
S → aA
A → bA | c

Ans)Recursive descent procedures
Grammar:
S → aA
A → bA | c

Procedures:

```
void S()
{
   match('a');
   A();
}
void A()
{
   if (lookahead == 'b')
   {
      match('b');
      A();
   }
   else if (lookahead == 'c')
   {
      match('c');
   }
   else
      error();
}
```

Q9. Given the grammar:
S → AB
A → a | ε
B → b
Construct the predictive parsing table.

Ans) Predictive parsing table
Grammar:
S → AB
A → a | ε
B → b
FIRST sets
FIRST(A) = {a, ε}
FIRST(B) = {b}
FIRST(S) = {a, b}
Predictive parsing table

Non-terminal	a	b	$
S	S → AB	S → AB	Error
A	A → a	A → ε	Error
B	Error	B → b	Error

Q10. Demonstrate the first three actions of a shift-reduce parser for:

id + id

Ans) First three actions of shift-reduce parser
Input:
id + id $
First three actions:

Step	Stack	Input	Action
1	$	id + id $	Shift id
2	$ id	+ id $	Reduce id → E
3	$ E	+ id $	Shift +

So the first three actions are:
Shift → Reduce → Shift

Q11. Identify the handle in the expression:
id * id + id
for the grammar:
E → E + E | E * E | id

Ans) Handle in id * id + id
Grammar:
E → E + E | E * E | id
Initially, the first reducible substring is:
id
So the first handle is:
id
After reduction:
id * id + id
↓
E * id + id
The next handle is another id.
Eventually:
E * E + E
and E * E becomes a handle for E → E * E.

Q12. Construct the operator precedence relations for the operators:
+, *
where * has higher precedence than +.

Ans)Operator precedence relations
Given:
* has higher precedence than +
Assuming both operators are **left associative**:

	+	*****
+	>	<
*****	>	>

Explanation:
- + < * → multiplication has higher precedence.
- * > + → multiplication is performed first.
- + > + → left associativity.
- * > * → left associativity.

Q13. Explain how operator precedence parsing handles the expression:
id + id * id

Ans)Operator precedence parsing of id + id * id
Expression:
id + id * id
Since * has higher precedence than +:
id + (id * id)
Parsing conceptually:
id
id * id → E
id + E → E
Therefore:
id + id * id
is interpreted as:
id + (id * id)

Q14. What are LR parsers? Illustrate with a simple parsing scenario.

Ans)What are LR parsers?
LR parsers are bottom-up parsers that scan the input from **Left to right** and construct a **Rightmost derivation in reverse**.
They use:

- A stack
- Input buffer
- ACTION table
- GOTO table

Example:
id + id
Basic actions:
Shift id
Reduce id → E
Shift +
Shift id
Reduce id → E
Reduce E + E → E
Accept
LR parsers can handle a large class of context-free grammars.

Q15. Given the grammar:
S → AA
A → aA | b
Write the steps involved in constructing an SLR parsing table.

Ans) Steps for constructing an SLR parsing table
Grammar:
S → AA
A → aA | b
Steps:

1. **Augment the grammar**

S' → S

2. Construct the **LR(0) item sets**.
3. Find the **closure** and **GOTO** of each item set.

4. Construct the **canonical collection of LR(0) items**.
5. Calculate the **FOLLOW sets**.
6. Construct the **ACTION table**:
    - Shift for terminal transitions.
    - Reduce using FOLLOW sets.
    - Accept for S' → S•.
7. Construct the **GOTO table** for non-terminals.
8. Check for **shift-reduce** or **reduce-reduce conflicts**.

Q16. Compute FOLLOW(A) for the grammar:
S → AB
A → a | ε
B → b

Ans) Compute FOLLOW(A)
Given:
S → AB
A → a | ε
B → b
Since A is followed by B:
FOLLOW(A) = FIRST(B)
And:
FIRST(B) = {b}
Therefore:
FOLLOW(A) = {b}

Q17. Construct the augmented grammar for:
S → aA
A → b

Ans)Augmented grammar
Given:
S → aA
A → b
Add a new start symbol S':
S' → S
S → aA
A → b
This is the **augmented grammar**.

Q18. Find the closure of the LR(0) item:
S' → •S
for the grammar:
S → aA
A → b

Ans)Closure of LR(0) item
Given:
S' → •S
Grammar:
S → aA
A → b

Since the dot is before S, add productions of S:
S' → •S
S → •aA
Now the dot is before terminal a, so no further items are added.
Therefore:
CLOSURE = {
S' → •S,
S → •aA
}

Q19. Check whether the grammar is ambiguous:
E → E + E | id
using the string:
id + id + id

Ans) Is the grammar ambiguous
Given:
E → E + E | id
String:
id + id + id
It has two possible parse structures:
(id + id) + id
and
id + (id + id)
Therefore, the grammar has **more than one parse tree** for the same string.
Hence, the grammar is ambiguous.

Q20. Construct a leftmost derivation for:
id + id * id
using:
E → E + E | E * E | id

Ans) Leftmost derivation
Given:
E → E + E | E * E | id
String:
id + id * id
Leftmost derivation:
E
⇒ E + E
⇒ id + E
⇒ id + E * E
⇒ id + id * E
⇒ id + id * id
Therefore:
E ⇒ E + E ⇒ id + E ⇒ id + E * E
⇒ id + id * E ⇒ id + id * id

Q21. Apply left factoring to:
S → aBc | aBd | b

Ans)Apply left factoring

Given:
S → aBc | aBd | b
Common prefix:
aB
Left-factored grammar:
S → aB S' | b
S' → c | d

Q22. Find FIRST(F) for:
F → (E) | id

Ans)Find FIRST(F)
Given:
F → (E) | id
The alternatives begin with (and id.
Therefore:
FIRST(F) = { (, id }

Q23. Determine whether the grammar is suitable for predictive parsing:
S → aA | aB
A → b
B → c
Justify your answer.

Ans)Is grammar suitable for predictive parsing
Given:
S → aA | aB
A → b
B → c
No, it is not directly suitable for predictive parsing.
Reason:
Both productions of S have the same FIRST symbol:
FIRST(aA) = {a}
FIRST(aB) = {a}
Therefore, the parser cannot decide which production to select using one lookahead symbol.
Apply left factoring:
S → aS'
S' → A | B
A → b
B → c
So the original grammar is **not suitable for LL(1) predictive parsing without left factoring**.

Q24. Construct recursive descent parser procedures for:
S → abS | ε

Ans)Recursive descent parser procedures
Grammar:
S → abS | ε
Procedure:
```
void S()
{
   if (lookahead == 'a')
```

```
    {
        match('a');
        match('b');
        S();
    }
    else if (lookahead == '$')
    {
        return;
    }
    else
    {
        error();
    }
}
```

Here, ε means the parser can return without consuming an input symbol.

Q25. Build the predictive parsing table for:
S → aS | b
and show the table entries for terminals a, b, and $.

Ans)Predictive parsing table
Grammar:
S → aS | b
FIRST sets
FIRST(aS) = {a}
FIRST(b) = {b}
Parsing table

Non-terminal	**a**	**b**	**$**
S	S → aS	S → b	Error

Therefore:
M[S,a] = S → aS
M[S,b] = S → b
M[S,$] = Error

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand